

SKILL_08

1. TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

CODE:

```
root@SRIVALLI: ~
GNU nano 7.2 combined_redirection.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int fd;

    fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if(fd < 0)
    {
        perror("File Open Error");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);

    printf("This is standard output.\n");
    fprintf(stderr, "This is an error message.\n");

    close(fd);

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano combined_redirection.c
root@SRIVALLI:~# gcc combined_redirection.c -o combined_redirection
root@SRIVALLI:~# ./combined_redirection
root@SRIVALLI:~# cat combined.txt
This is an error message.
This is standard output.
```

REPORT:

In this experiment, both standard output (stdout) and standard error (stderr) were redirected to a single file using the dup2() system call. The program merged the two output streams and stored them in one file. This helps in understanding file descriptor duplication, stream redirection, and handling combined outputs in Linux systems.

2. To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax,Generate Execution Plans, Test Scenarios

CODE:

```
GNU nano 7.2                                complex_command.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pipe(fd);

    if(fork() == 0)
    {
        int out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[0]);
        close(fd[1]);

        execlp("ls", "ls", NULL);
        exit(1);
    }

    if(fork() == 0)
    {
        int out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

        dup2(fd[0], STDIN_FILENO);
        dup2(out, STDOUT_FILENO);

        close(fd[1]);
        close(fd[0]);

        execlp("sort", "sort", NULL);
        exit(1);
    }
}

close(fd[0]);
close(fd[1]);

wait(NULL);
wait(NULL);

printf("Output stored in output.txt\n");

return 0;
```

RESULT:

```
This is an error message.
This is standard output.
root@SRIVALLI:~# nano complex_command.c
root@SRIVALLI:~# gcc complex_command.c -o complex_command
root@SRIVALLI:~# ./complex_command
Output stored in output.txt
root@SRIVALLI:~# cat output.txt
My File.txt
OSSP
SkillWeek1
abc
append_redirection
append_redirection.c
builtin_shell
builtin_shell.c
client
client.c
combined.txt
combined_redirection
combined_redirection.c
command_executor
command_executor.c
command_executor.c.save
complex_command
complex_command.c
demo.fifo
dispatch_shell
dispatch_shell.c
double_quotes.c
dynamic_memory.c
error.txt
escape_sequences.c
export_env
export_env.c
fifo1
fifo2
file1.txt
fork_process
fork_process.c
history
```

REPORT:

In this experiment, a program was developed to execute complex commands using pipes and output redirection. The output of one command was passed to another command through a pipe, and the final result was redirected to a file. The program demonstrates command parsing, execution order, process communication, syntax handling, and output redirection. This experiment helps in understanding how shells execute complex commands involving both pipes and redirections in Linux systems.

3. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks
- CODE:

```
root@SRIVALLI: ~  
GNU nano 7.2 background_job.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <stdlib.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
  
    pid = fork();  
  
    if(pid == 0)  
    {  
        printf("Background Process Started\n");  
  
        sleep(10);  
  
        printf("Background Process Completed\n");  
    }  
    else  
    {  
        printf("Parent Process Continues...\n");  
        printf("Background PID: %d\n", pid);  
  
        wait(NULL);  
    }  
  
    return 0;  
}
```

RESULT:

```
root@SRIVALLI:~# nano background_job.c  
root@SRIVALLI:~# gcc background_job.c -o background_job  
root@SRIVALLI:~# ./background_job  
Parent Process Continues...  
Background PID: 1315  
Background Process Started  
Background Process Completed
```

REPORT:

In this experiment, a background process was created using the fork() system call. The child process executes independently while the parent process continues without blocking. The process ID of the background task is stored and displayed for monitoring. This experiment helps in understanding background job execution, process management, and non-blocking operations in Linux systems.

4. To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

CODE:

```
root@SRIVALLI: ~  
GNU nano 7.2 job_table.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
struct Job  
{  
    int pid;  
    char state[20];  
};  
  
int main()  
{  
    struct Job jobs[5];  
    int count = 0;  
  
    for(int i = 0; i < 3; i++)  
    {  
        pid_t pid = fork();  
  
        if(pid == 0)  
        {  
            sleep(5);  
            return 0;  
        }  
        else  
        {  
            jobs[count].pid = pid;  
            sprintf(jobs[count].state, "Running");  
        }  
    }  
}
```

```
root@SRIVALLI: ~  
GNU nano 7.2 job_table.c *  
  
printf("\nJob Table:\n");  
  
for(int i = 0; i < count; i++)  
{  
    printf("PID: %d State: %s\n",  
        jobs[i].pid,  
        jobs[i].state);  
}  
  
for(int i = 0; i < count; i++)  
{  
    wait(NULL);  
    sprintf(jobs[i].state, "Completed");  
}  
  
printf("\nUpdated Job Table:\n");  
  
for(int i = 0; i < count; i++)  
{  
    printf("PID: %d State: %s\n",  
        jobs[i].pid,  
        jobs[i].state);  
}  
  
return 0;  
}
```

RESULT:

```
root@SRIVALLI:~# nano job_table.c  
root@SRIVALLI:~# gcc job_table.c -o job_table  
root@SRIVALLI:~# ./job_table  
  
Job Table:  
PID: 1412 State: Running  
PID: 1413 State: Running  
PID: 1414 State: Running
```

REPORT:

In this experiment, a job table was created to store and manage process information. Multiple child processes were created using `fork()`, and their process IDs and states were stored in the job table. The program tracked the status of each job, updated the state when execution completed, and maintained accurate job records. This experiment helps in understanding job management, process tracking, state updates, and job control in Linux systems.